Networking: The fastest tour

Putting it together before we take it all apart

Ian Batten, September 2024

A new approach

It helps to know what a car does before we discuss understeer

- I've tried this module both ways: from the cables up to the applications, and from applications down to cables.
- Both work, but there's a lot of "you'll see why this matters later"
- What's needed is a general overview, on which we can then build.
- I've said in the past that my objective is to be able to give a detailed answer to the classic interview question, "you click on a link, what happens next? Next? Next?"
- So I'm going to aim to do that today and (probably) next lecture.

The legendary Unix kernel comment

```
2230 /*
2231  * If the new process paused because it was
2232  * swapped out, set the stack level to the last call
3333  * to savu(u_ssav).  This means that the return
2235  * actually returns from the last routine which did
2236  * the savu.
2237  *
2238  * You are not expected to understand this.
2239  */
2240 if(rp->p_flag&SSWAP) {
2241   rp->p_flag =& ~SSWAP;
2242   aretu(u.u_ssav);
2243 }
```

I am intending to give a general flavour of what happens, so that as we go through the details in following lectures you have a framework into which to plug things.

Glossary

- Hosts, systems, computers: things on which we run useful programs
  - not (even though sometimes they share hard- and software) routers or switches.
- Switches: things that move packets inside “subnets” or “LANs”
  - where you can in the limit broadcast a packet and assume the recipient will see it.
- Routers: things that move packets between networks,
  - when you need more knowledge of the topography to get packets to the right places
- The switch router boundary is today unclear, especially as routers almost always have a switch fabric attached.

What does happen when you click on a link?

- Let's consider https://www.bbc.co.uk/index.html
  - Parse the URL
  - Use DNS to find the IP address of the destination
  - Make a TCP connection
  - Some crypto stuff we will wave our hands over
  - Fetch content with http
- At each point, we need to be able to send and receive data on networks

Parse the URL

scheme://host/path

- We have the URL https://www.bbc.co.uk/index.html
- scheme: the protocol or other mechanism we are going to use. **https** is the HyperText Transfer Protocol, Secure. Others would include **http** (not encrypted), **file** (local files), **imap** or **imaps** (email reading), **mailto** (email sending), obsolete stuff like **ftp**, **gopher** and others.
- The host and path section are specific to the scheme, and may have a different syntax (eg, “mailto:i.g.batten@bham.ac.uk" has no slashes).
- In this case, it’s a host or server or machine (between // and the first /) and then a path interpreted on that machine (after the first single slash)
- It looks like a Unix path because it originally was a Unix path, but the server can do what it wants with it.

DNS

The Domain Name Service: the oldest and scariest bit

- The DNS is one of the most mysterious parts of the Internet. A friend of mine made a living, to the point of flying Emirates first class as a condition of employment, fixing obscure DNS problems.
- For our purposes here, it maps names into IP addresses
- The means of operation are complex, and contain a lot of extra facilities to make life easier on the (slow, unreliable, intermittently connected) Internet of forty years ago. It was codified in **RFC 1034** and **RFC 1035** (October 1987) and has not substantially changed since. Implementations have (although not as much as we'd like)

Why do we need the DNS

IP Addresses

- We will be using the Internet Protocol (IP, **RFC791**) to move data around.
- IP requires computers and other devices to have “IP Addresses”
- There are two versions of IP in current use: IPv4 and IPv6
- IPv4 addresses are 32 bits, conventionally notated as four bytes, each encoded in decimal and separated with dots.
  - For example: **74.125.133.147**
- IPv6 addresses are 128 bits, conventionally notated as 32 hexadecimal digits in eight groups of four, separated by colons, leading zeros in each group suppressed, with :: meaning “as many zeros as necessary to make it up to the right length:
  - For example: **2a00:1450:400c:c07::6a** is short for **2a00:1450:400c:c07:0:0:0:67**

Why IPv4 and IPv6

- IPv4 addresses are 32 bits long, so ~4 billion addresses
- Population of the planet is ~8 billion, and in the developed world we consume at least one address each.
- In reality, addresses were very inefficiently and unfairly allocated, surplus in US, probably about OK in Europe, terrible shortage elsewhere. A small number of ex-NATO companies and universities have 16 million addresses each, rather more have 65536 each, elsewhere it can be hard to get one.
- Estimates vary, but it could easily be 90% of potential addresses being either unusable or unreachable.
- IPv6 is 128 bit addresses, with the basic unit of allocation being a 64 bit “prefix”. That means there are ~2 billion 64-bit prefixes for every person in the world.
- In general (we will talk about this later) the left hand end of an IP address specifies a network, the right hand end an entity on that network. Where the dividing line is can be complex.

Why the DNS?

- You might imagine a very small internet in which you periodically distribute a file containing mnemonic names and IP addresses. This was “HOSTS.TXT”, which lives on in /etc/hosts and similar names on various operating systems.
- You can just about make sense of it (VAXes, Pyramids, Sun workstations are Unix machines of a lost age; Multics is the great progenitor of all Unix-y operating systems, which the very keen could now run in emulation on a Raspberry Pi)
- For those that understand IP address “classes”, note there is **nothing** between 128.18 and 192.5.1.
- The size and rate of change makes this mechanism untenable

```
HOST : 128.6.0.194 : TOPAZ,RU-TOPAZ : PYRAMID-90X : UNIX : TCP/TELNET,TCP/FTP,TCP/SMTP :
HOST : 128.6.0.195 : OPAL,RU-OPAL : SUN-150 : UNIX : TCP/TELNET,TCP/FTP,TCP/SMTP,TCP/FINGER :
HOST : 128.18.0.201 : SRI-TSCA,TSCA : VAX-11/750 : UNIX : TCP/TELNET,TCP/FTP,TCP/SMTP,ICMP,TCP/TIME,TCP/FINGER :
HOST : 128.18.0.202 : SRI-JOYCE,JOYCE,SRIJOYCE : VAX-11/750 : UNIX : TCP/FTP,TCP/TELNET,TCP/SMTP,UDP,TCP/TIME,TCP/FINGER :
HOST : 192.5.1.0 : CISL-DEVELOPMENT-MULTICS : H-68/80 : MULTICS : TCP/TELNET,TCP/FTP,TCP/SMTP,TCP/TIME,TCP/FINGER,TCP/ECHO,TCP/DISCARD :
HOST : 192.5.2.1 : WISC-RSCH,RSCH,UWVAX : VAX-11/780 : UNIX : TCP/TELNET,TCP/FTP,TCP/SMTP,TCP/ECHO,TCP/FINGER,ICMP :
HOST : 192.5.8.1 : UW-BEAVER,BEAVER,THEODORE : VAX-11/750 : UNIX : TCP/TELNET,TCP/FTP,TCP/SMTP :
```

Or even no names…

- You could even just about imagine a world in which google.com is reached by typing **http://74.125.133.103/** and you just need to remember it. Not nice, though.
- **http://[2a00:1450:400c:c07::67]/** is even less pleasant
- I think we're all pretty clear we need memorable or descriptive names, rather than raw addresses.
- The DNS does this job.
- If we can translate a hostname into an IP address, we can send some data to that address as though we were sending it to the name.

Port Numbers

- You'll have noticed in the HOSTS.TXT format notations like:
  - `TCP/TELNET,TCP/FTP,TCP/SMTP,ICMP,TCP/TIME,TCP/FINGER`
- You might recognise some of these: SMTP is still used to ship email, you've probably heard of Telnet (terminal emulation) and FTP (file transfer protocol).
- These are distinct services, offered by distinct pieces of software, and we need to be able to address them distinctly: "here's some email", "please let me run some commands", "I want to transfer files".

Well Known Port Numbers

- Each service has a **port number** assigned to it. This is a 16 bit quantity specifying an endpoint on a computer (there's more to it than this, but this will do for now).
- 21 is telnet, 23 is ftp, 25 is smtp…53 is DNS. 22 is ssh, 80 is http.
  - For historical reasons, "old" protocols have odd port numbers, "newer" protocols filled in the even ones. If you want to know why, you'll need to explore the pre-1983 NCP protocol.
  - /etc/services on Linux/MacOS boxes has the whole list, much of it obsolete.

OK, so how do we look up a name?

- The Domain Name Service lives on port 53.
- Our machine is pre-configured (we'll talk about how when we talk about DHCP, amongst other things) to know where it can send queries.
- We choose a random high port number to receive replies, and send the question "what is the IP address of www.bbc.co.uk" from that port to a pre-defined IP address, port 53.
- That system does some magic to find the answer, and sends it back to us.

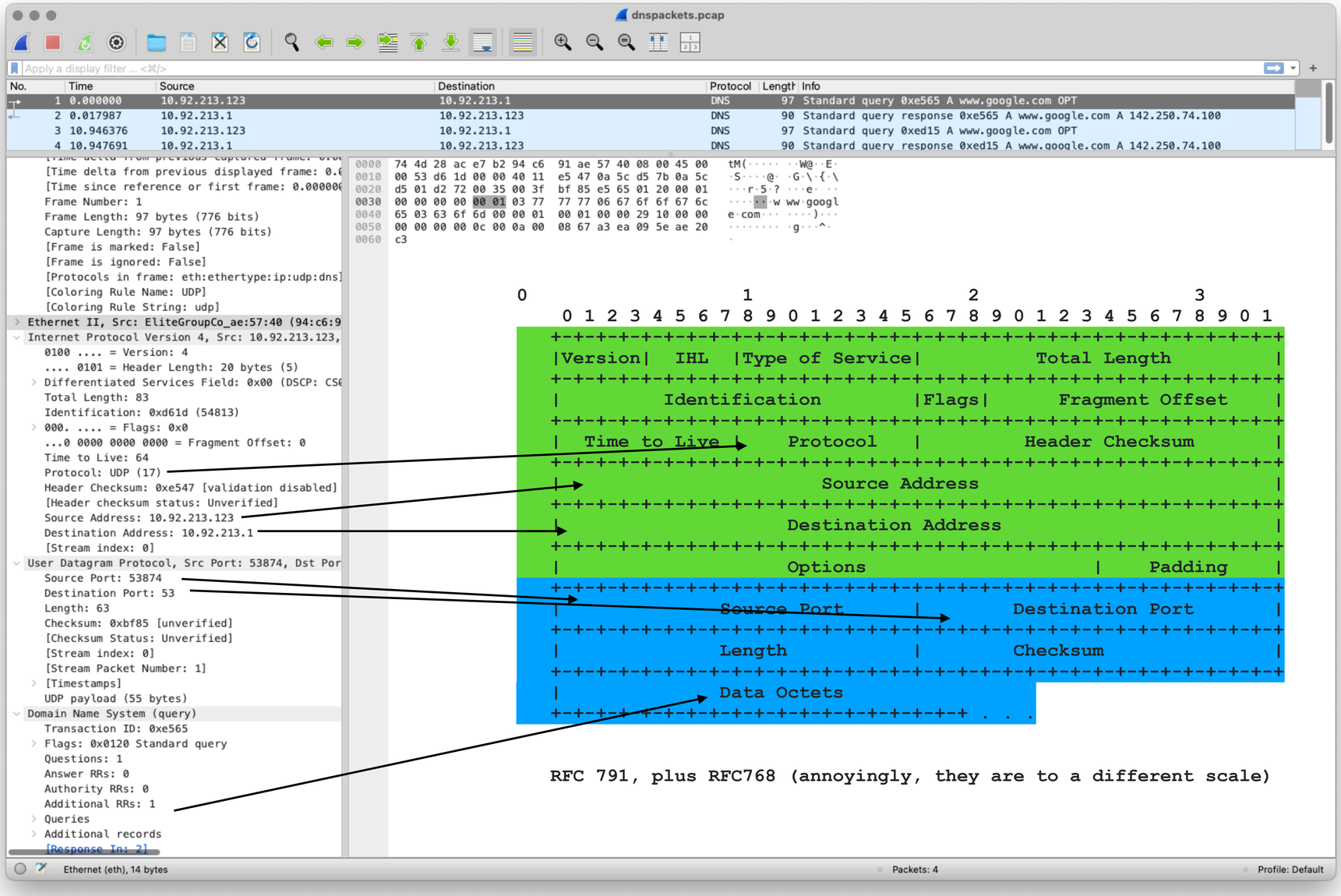
UDP versus TCP

- When I say “send the question”, how do we do it?
- Although there are others we will touch on, there are two basic “transport protocols” in use.
- A transport protocol is something a program can use to send data to another program, by sending it to a host with an annotation (in this case port numbers) to ensure it goes to the right place.
- UDP (**RFC768**) is “send an unreliable packet and hope it gets there.”
- TCP (**RFC9293**, but classically **RFC793**) does more work both initially and during transmission to offer guarantees of reliability and error checking.

DNS, in this case, is conventionally over UDP

- There's a whole debate about whether this is the right choice in 2024, but today DNS "resolution" (the process of looking up addresses for names) is mostly done over UDP. We can come to TCP when we talk about HTTP.
  - The idea is that UDP is better for single question, single answer situations, whereas building a TCP connection — think "BufferedStream" in Java terms — is more expensive.
- We load into a packet our IP address, a random port number, the resolver's IP address (8.8.8.8, or in my case 10.92.213.1), the resolver's port number (port 53) and the question itself.
- We then send the packet. How?

We can look at packets with Wireshark



Sending a packet to 10.92.213.1

- For Linux, the command “ip addr” shows the address of an interface

```
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 94:c6:91:ae:57:40 brd ff:ff:ff:ff:ff:ff
    altname enp0s31f6
    inet 10.92.213.123/24 brd 10.92.213.255 scope global eno1
       valid_lft forever preferred_lft forever
```

- The 10.92.213.123 looks like an IP address, but what does the /24 (pronounced “slash twenty-four”) mean?
- It tells us what’s network and what’s host. 24 bits specify the network (“10.92.213”) and “123” is one host out of a possible 254 (“all zeros” and “all ones” are reserved).

So the packet is “local”

- We are sending to 10.92.213.1, and we know that our current network is 10.92.213.0/24. So the packet is “local”.
- The /24 is a “net mask”: it’s the number of bits of the address, counting from the left, that you consider to be the network rather than the host on the network.
- Equality is after the mask has been applied, with /24 meaning 1*24 0*8.
  - 10.92.213.1 & 255.255.255.0 == 10.92.213.123 & 255.255.255.0
  - 0x0a5cd501 & 0xffffff00 == 0x0a5cd57b & 0xffffff00
- IPv6 prohibits masks that are not 4-bit (one hex digit) aligned: IPv4 does not, and masks like /28, /23 and /21 are routinely used.

How do we send an IP packet on an ethernet?

- Let's assume our devices are connected by something that looks like an ethernet, wireless or wired or fibre. There are not-Ethernet protocols, and we will talk about some, but ethernet is the basic building block.
  - Not-Ethernet is used to encapsulate IP the same as Ethernet: you place an IP packet into a not-Ethernet packet, and use the not-Ethernet address to send it to the next place.
- Devices have (at least) two addresses: an IP address, which is a structured address allocated by means we will come to. And an ethernet address, which is in general assigned either randomly or at manufacture. In 2024, it should be unique per interface.
- In order to send an IP packet to a destination, we need to know the ethernet address of the owner of that IP address. We then encapsulate the IP packet in an ethernet frame with the correct addresses.

ARP: the first place I say “don’t do crimes”

- A regular fixture in my lectures will be “don’t do crimes”: I will either describe, or it will be obvious to you, things which permit the interception or otherwise illegal manipulation of network traffic.
- It is not a defence to say that had the sender wanted their traffic to be secure, they shouldn’t have sent it like that. Don’t do crimes.
- ARP is very crude: Broadcast a packet saying “who has this IP address”, hope that someone (probably the target itself) replies. Here is 10.92.213.123 looking for 10.92.213.44:

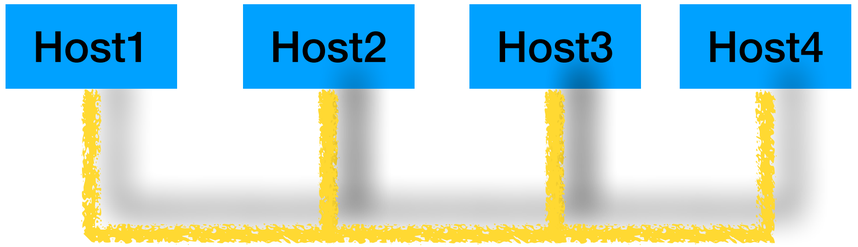
ARP and Ethernet

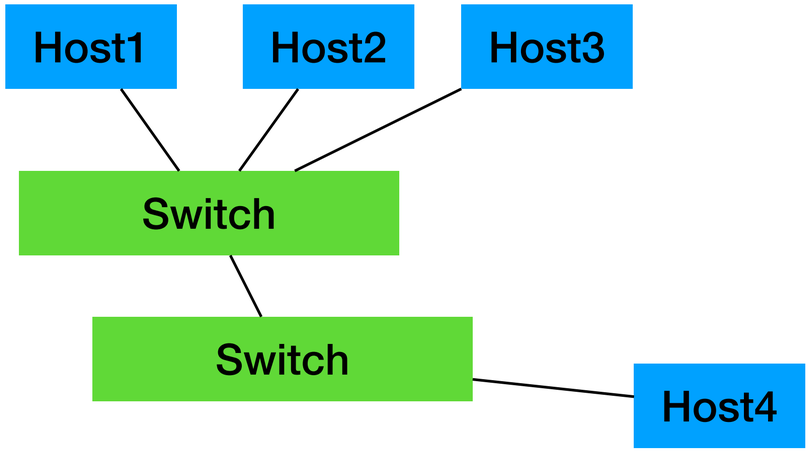
- IPv4 addresses, which is what we are using here, are 32 bits.
- Ethernet addresses are 48 bits (conventionally a 24 bit vendor code and a 24 bit random-y bit, but don't rely on that).
- Here is 10.92.213.123 "arp-ing" for 10.92.213.44:

```
10:44:18.362837 ARP, Request who-has 10.92.213.44 tell 10.92.213.123, length 28
10:44:18.363097 ARP, Reply 10.92.213.44 is-at d4:ca:6d:ec:5d:b2 (oui Unknown), length 46
```

Now we can send the packet

- Both devices will be, in 2024, connected to an ethernet switch or multiple switches.
- An ethernet was originally one single cable joining the whole group of machines together, to which all packets were broadcast. Broadcasts were processed by everyone, unicast was processed only by the host to which it was addressed (but they saw everything).
- An ethernet switch figures out which port each device is on, by looking at its transmissions, and where it can limits transmission to just the relevant ports.
- But the senders do not need to know anything about this: they can pretend it's just one large collection of machines



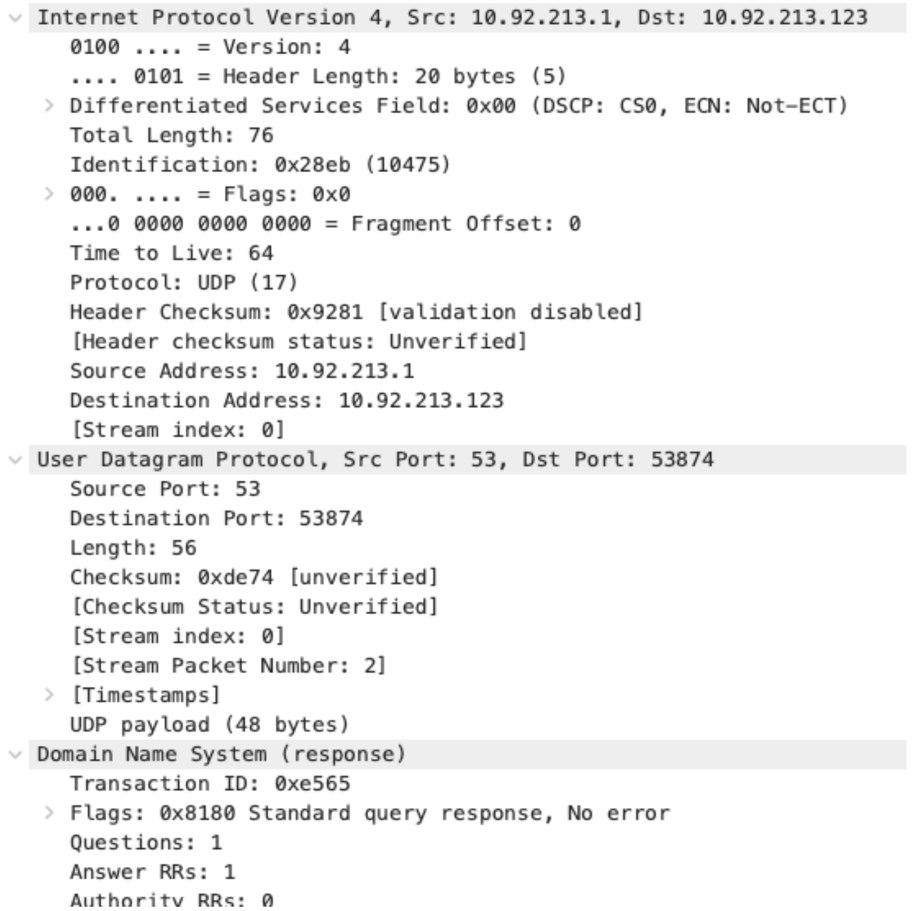


Ethernet Encapsulation

```
Frame 1: 97 bytes on wire (776 bits), 97 bytes captured (776 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Sep 11, 2024 09:21:50.790320000 BST
    UTC Arrival Time: Sep 11, 2024 08:21:50.790320000 UTC
    Epoch Arrival Time: 1726042910.790320000
    [Time shift for this packet: 0.000000000 seconds]
    [Time delta from previous captured frame: 0.000000000 seconds]
    [Time delta from previous displayed frame: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 97 bytes (776 bits)
    Capture Length: 97 bytes (776 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:udp:dns]
    [Coloring Rule Name: UDP]
    [Coloring Rule String: udp]
Ethernet II, Src: EliteGroupCo_ae:57:40 (94:c6:91:ae:57:40), Dst: Routerboardc_ac:e7:b2 (74:4d:28:ac:e7:b2)
    Destination: Routerboardc_ac:e7:b2 (74:4d:28:ac:e7:b2)
    Source: EliteGroupCo_ae:57:40 (94:c6:91:ae:57:40)
    Type: IPv4 (0x0800)
    [Stream index: 0]
Internet Protocol Version 4, Src: 10.92.213.123, Dst: 10.92.213.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 83
```

The DNS resolution then takes place

- How the local resolver finds the IPv4 or IPv6 address of the name we are interested in is another topic, but let's assume that it "just knows".
- It then sends a reply by the same means: load it into a UDP frame with a source port of 53, a destination port of 53874 (the sender's random choice), and then send it via Ethernet.



So now we know the IP address of www.bbc.co.uk

- So now we know that www.bbc.co.uk can be reached on 212.58.235.1.
- The DNS response for www.bbc.co.uk is actually quite complex, which we'll discuss later. But the IP addresses are pretty clear.

```
www.bbc.co.uk.                  3600    IN      CNAME   www.bbc.co.uk.pri.bbc.co.uk.
www.bbc.co.uk.pri.bbc.co.uk.    300 IN      CNAME   gtm-live.pri.bbc.co.uk.
gtm-live.pri.bbc.co.uk.         300     IN      A       212.58.235.1
gtm-live.pri.bbc.co.uk.         300     IN      A       212.58.236.129
gtm-live.pri.bbc.co.uk.         300     IN      A       212.58.237.1
gtm-live.pri.bbc.co.uk.         300     IN      A       212.58.236.1
gtm-live.pri.bbc.co.uk.         300     IN      A       212.58.237.129
gtm-live.pri.bbc.co.uk.         300     IN      A       212.58.235.129
```

What are we going to do?

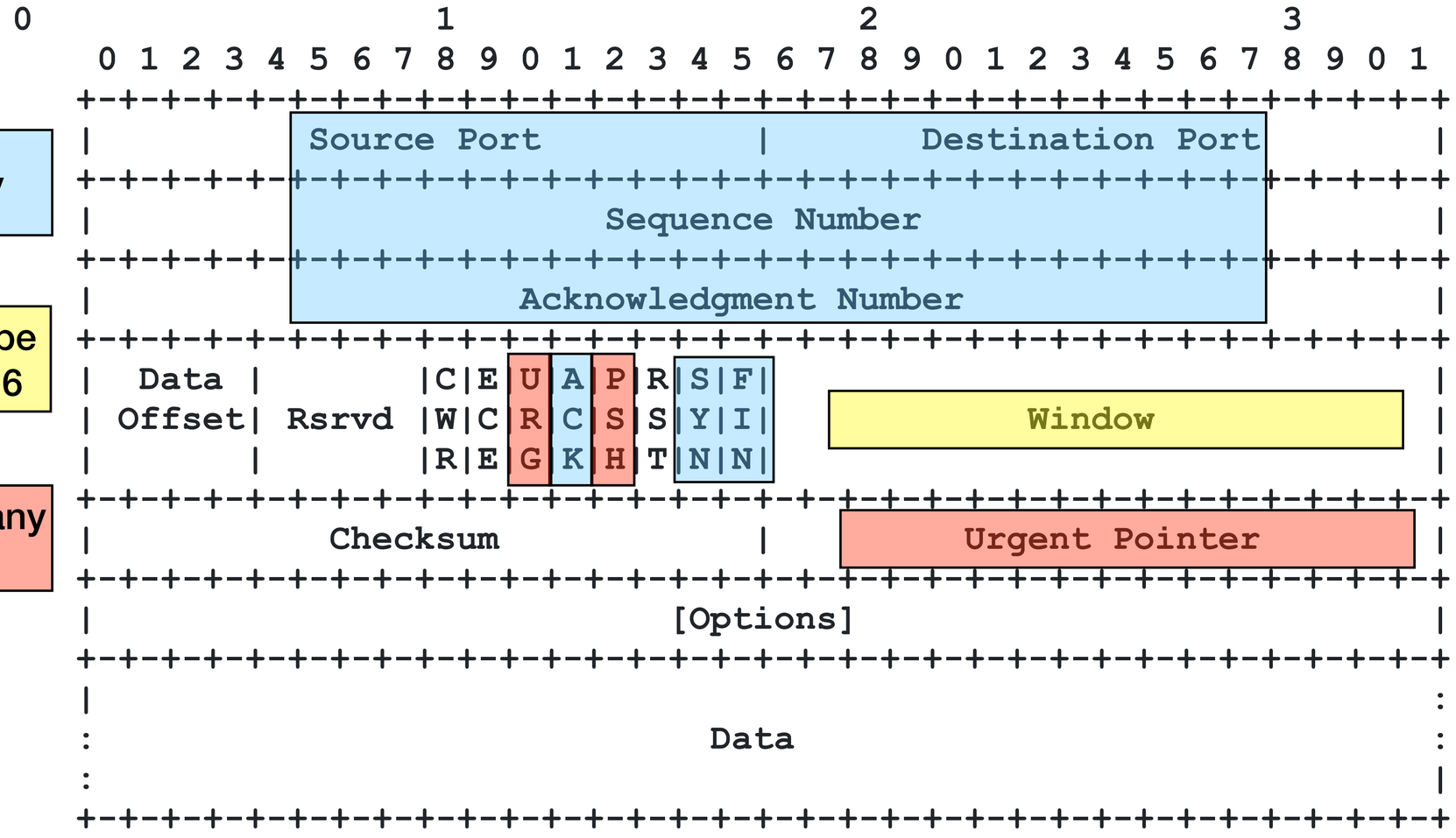
- We're going to make a "TCP connection" to www.bbc.co.uk's web server software at 212.58.235.1, and talk HTTP over it.
  - You can talk HTTP to port 80, or HTTPS (encrypted HTTP) to port 443.
- TCP is more complicated than UDP. There is work to do to set the connection up, so I am going to wave my hands here. The nuances of TCP will be a couple of weeks of this module. I am leaving out a **lot** of detail.
- We are also going to talk to 212.58.235.1, which is clearly not local.

TCP Frames

Stuff for today

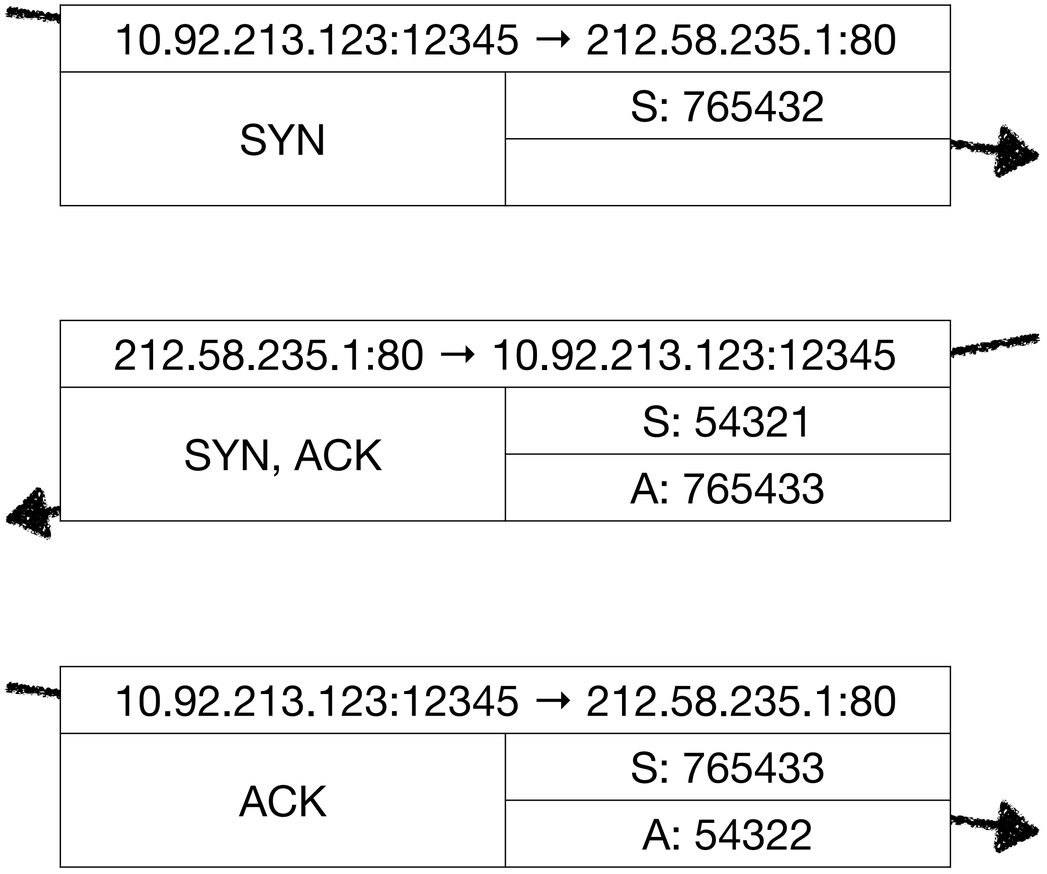
Sadly needs to be 32 bits, but is 16

Not really used any more



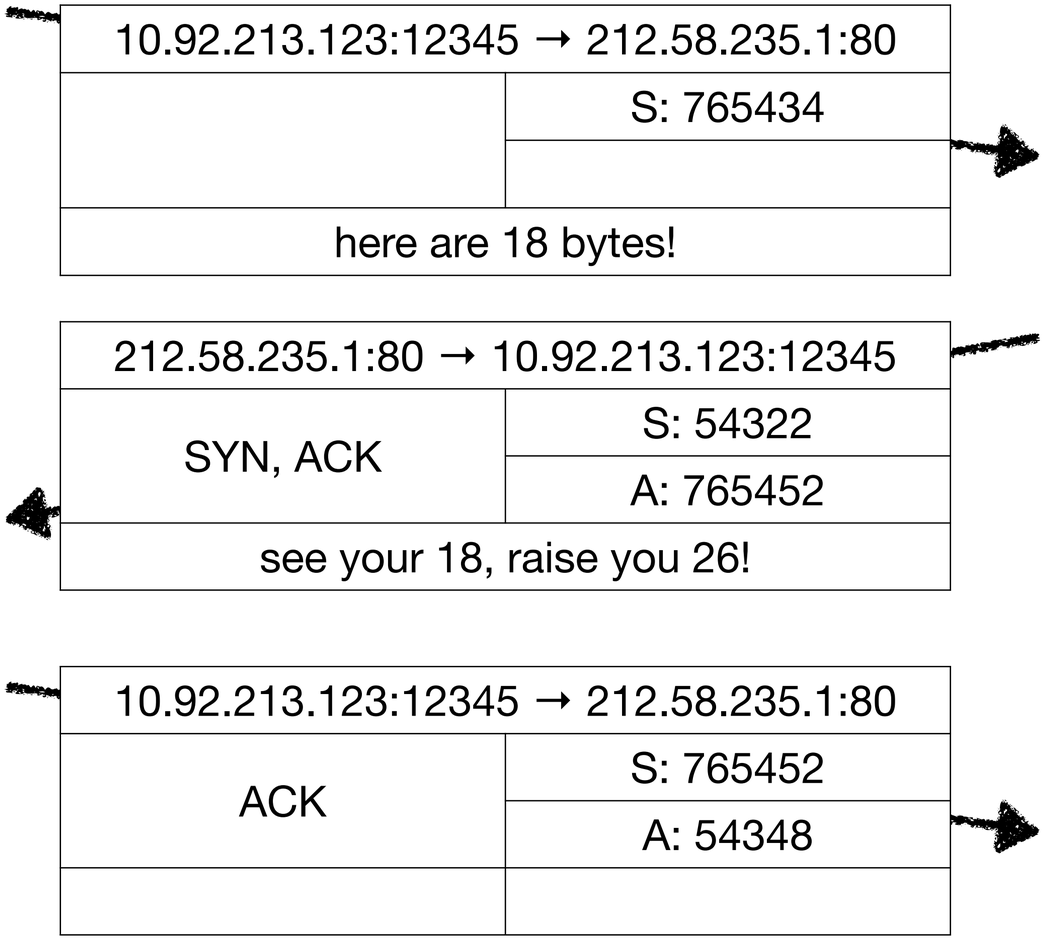
TCP setup, in a slide

- A→B: I am going to send a stream of bytes, numbered from this initial sequence number, from this port number to this port number. This is called SYNchronise
- B→A: OK, got that. I am going to send a stream of bytes, numbered from this initial sequence number, back to you. This is called SYNchronise ACKnowledge.
- A→B: OK, got that. ACKnowledge



TCP bulk data, in a slide

- A→B: I got your last, and here's some bytes.
- B→A: I got your last, and here's some bytes.
- A→B: Nothing for you, but I got yours.
- The ACK field is the next sequence number expected.



OK, but how to we get traffic to 212.58.235.1?

- For traffic which is **not** local, how do we get there? 212.58.235.1 is not on 10.92.213.0/24.
- Your machine has routing table, which tells it how to reach remote systems.

```
default via 10.92.213.1 dev eno1 onlink
10.92.213.0/24 dev eno1 proto kernel scope link src 10.92.213.123
81.187.150.208/28 dev eno1.5 proto kernel scope link src 81.187.150.212
169.254.0.0/16 dev eno1 scope link metric 1000
```

- This machine has two interfaces, one via a VLAN, which we will come to later (81.187.150.208/28, a network of 14 machines, is also local to it)
- We're actually looking at "default".

What does this mean?

```
default via 10.92.213.1 dev eno1 onlink
10.92.213.0/24 dev eno1 proto kernel scope link src 10.92.213.123
81.187.150.208/28 dev eno1.5 proto kernel scope link src 81.187.150.212
169.254.0.0/16 dev eno1 scope link metric 1000
```

- A routing table is a list of network/mask pairs with an interface/gateway pair, and in more complex cases can map the entire Internet (2^{24} entries, 16777216 – easily fits in RAM on any remotely modern machine).
- For example, `10.92.213.0/24 dev eno1` means “if you have an address whose first 24 bits are 10.92.213 use interface eno1 directly via arp” and similarly for 81.187.150.208/28 (work out which addresses that contains)
- But it’s the “default” entry we’re most interested in. Note it says `via 10.92.213.1 dev eno1`. What does the “via” mean?

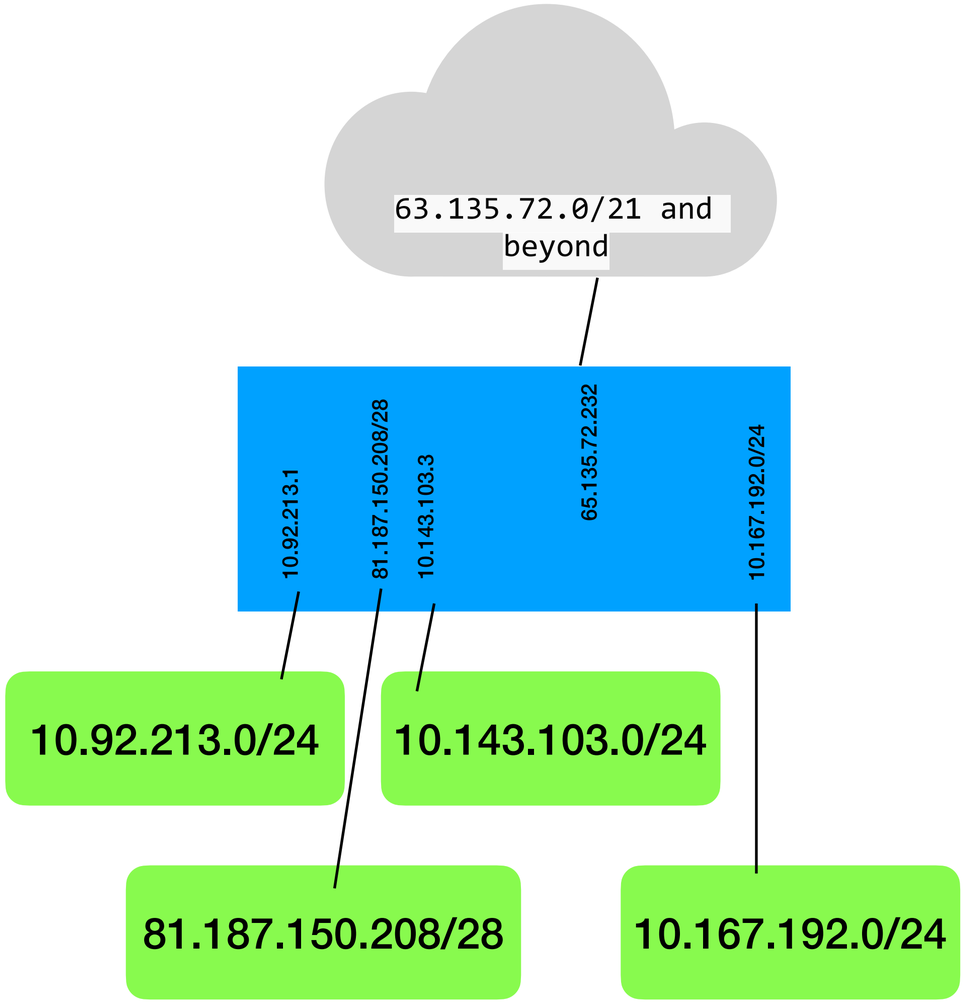
Routers

- Devices connected to the same ethernet (and we'll see next week why an Ethernet switch is "the same" network) can communicate by simply loading the address into a packet and throwing it out: it'll get there.
- But when networks are more spread out, we need entities called "routers". They connect to multiple networks and pass packets between them.

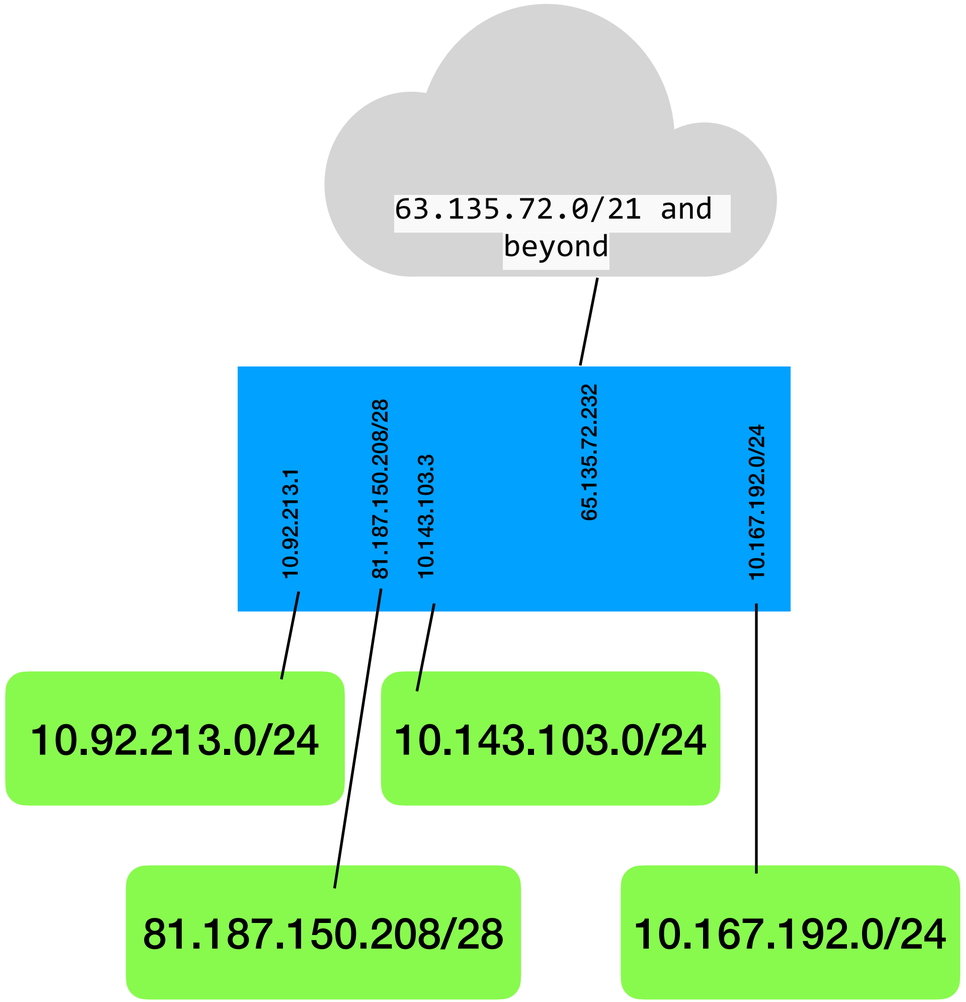
Addresses and routing table on a real router

```
0   81.187.150.213/28  81.187.150.208  Redzone5
1   10.92.213.1/24     10.92.213.0     OldDefaultNetwork
2   10.143.103.3/24    10.143.103.0    WiredGuest0011
3   10.167.192.2/24    10.167.192.0    WirelessGuest0009
4 D 63.135.72.232/21   63.135.72.0     BRSK2001
```

```
Flags: D - DYNAMIC; A - ACTIVE; c - CONNECT, s - STATIC, d - DHCP
Columns: DST-ADDRESS, GATEWAY, DISTANCE
#     DST-ADDRESS        GATEWAY             DISTANCE
  DAd 0.0.0.0/0          63.135.72.1                1
  DAc 10.92.213.0/24     OldDefaultNetwork          0
  DAc 10.143.103.0/24    WiredGuest0011             0
  DAc 10.167.192.0/24    WirelessGuest0009          0
  DAc 63.135.72.0/21     BRSK2001                   0
```



- A device on one of the green networks can access the outside world by sending traffic to the router, in blue, and then letting it sort it out.
- In this simple case, the router itself has a default route of 63.135.72.232 which is (presumably) another router.
- 63.135.72.20/21 is a network of 2^{13} (8096) devices: in this case, a suburban Gigabit PON. Yes, my wife and I rather pointlessly have symmetric GigE at home.



At last: send a packet to 212.58.235.1

- It's not local
- Look in the routing table: is there a match? No? Is there a default route?
  - You'll sometimes see the default route notated as 0.0.0.0/0: why?
- If we have "via" or "gateway", load an IP packet addressed to the **destination** (in this case 212.58.235.1) into an ethernet frame and send it to the **gateway** using ARP.
- It'll know what to do with it. In this case, the router ARPs for 63.135.72.1 and sends it there. The IP header address does not change, the ethernet address is always local to the link it's being sent on.

This is saying “I don’t like sending unencrypted content, go and get the encrypted version” both now, and for the next 365 days too (365*86400)

```
* Host www.bbc.co.uk:80 was resolved.
* IPv6: (none)
* IPv4: 212.58.235.1, 212.58.237.129, 212.58.235.129, 212.58.236.129, 212.58.237.1, 212.58.236.1
*   Trying 212.58.235.1:80...
* Connected to www.bbc.co.uk (212.58.235.1) port 80
> GET /index.html HTTP/1.1
> Host: www.bbc.co.uk
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 301 Moved Permanently
< Date: Wed, 11 Sep 2024 14:35:57 GMT
< Content-Type: text/html
< Content-Length: 166
< Connection: keep-alive
< Keep-Alive: timeout=120
< Location: https://www.bbc.co.uk/index.html
< Strict-Transport-Security: max-age=31536000; preload
<
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>openresty</center>
</body>
</html>
```

This blank line matters and must be sent

This blank line matters and will be sent

We sent this http request

This is the http response

This is 166 bytes of content, here HTML

TCP tear-down, in a slide

- A→B: I'm done now
- B→A: Yeah, got that.
- B→A: And now I think about it, I'm done too.
- A→B: Got that